



Nome del progetto	Piattaforma Cochise
Documento	Linee guida rilascio applicazioni su Cochise

Stato del documento	Revisione sezioni
Versione	1.0.0
Data	04/2025



Sommario

1. Introduzione	3
1.1. Obiettivi del documento	3
2. Linee guida per il deploy di applicazioni su piattaforma evoluta Cochise2.....	3
2.1. Regole di base per il dispiegamento di microservizi	4
3. Monitoraggio e Logging	7
3.1. OpenTelemetry.....	7
4. Gestione aperture firewall di servizi esterni e Database	8
5. Gestione comunicazioni asincrone.....	8
5.1. Modelli di comunicazione: sincrono e/o asincrono	8
6. Primo deploy su Cochise2.....	9
7. Autodeploy su piattaforma Cochise2	10
7.1. Implementazione e Pubblicazione immagini sul registry evoluto.....	11
7.2. Naming convention per la produzione delle immagini di progetto	12



1. Introduzione

Obiettivo del presente documento è quello di descrivere le linee guida di Regione Toscana per il deploy e l'aggiornamento di applicazioni all'interno della piattaforma evoluta Cochise 2 basata su Kubernetes.

1.1. Obiettivi del documento

Come premessa, poiché la piattaforma Cochise nasce e si è evoluta come piattaforma che ospita Applicazioni a microservizi, si raccomanda fortemente un approccio alla progettazione di applicazioni a microservizi al fine di sfruttare le caratteristiche principali di tali architetture che riassumiamo brevemente nei paragrafi successivi.

2. Linee guida per il deploy di applicazioni su piattaforma evoluta Cochise2

Di seguito vengono descritte alcune delle linee guida per il deploy sulla nuova piattaforma Cochise 2.

Nella seguente tabella riassumiamo i limiti di memoria impostabili sulla piattaforma per ogni singolo microservizio da dispiegare.

Linguaggio/Framework	Limite max memoria	Limite Max estensione
Java/Spring	384 MB	1 GB
Java/Quarkus	256 MB	1 GB
NodeJS	256 MB	1 GB



Golang	64 MB	384 MB
--------	-------	--------

NOTA: Il limite max di estensione va concordato sempre con il referente regionale della Piattaforma Cochise 2 e deve essere motivato.

NOTA: È fortemente consigliato l'utilizzo di linguaggi di sviluppo a basso impatto di risorse, come Golang.

NOTA: Nel caso di microservizio sviluppato in java, è obbligatorio impostare nel Dockerfile la variabile *javaMemory* come mostrato nell'esempio. In questo modo, i parametri della JVM sono controllabili direttamente dall'infrastruttura ed è possibile modificarne i valori.

Il valore di default del parametro *javaMemory* per applicativi java con Spring è 256MB, mentre quelli implementati mediante Quarkus è 128MB.

È vietato impostare i parametri della JVM direttamente nel Dockerfile, in ogni caso non saranno accettati deployment che violano le regole di memoria sopra definite.

2.1. Regole di base per il dispiegamento di microservizi

Di seguito alcune delle best practice che devono essere adottate per lo sviluppo di Applicazioni dispiegabili sulla piattaforma Cochise2.

- Il front end deve essere necessariamente deployato su un front-end nativo (apache e/o nginx) come, ad esempio, servizi.toscana.it o lavoro.toscana.it facendo esplicita richiesta ai referenti del front end di riferimento. Altri scenari di dispiegamento dovranno essere concordati con il referente regionale della piattaforma e con il referente regionale dell'applicazione.
- L'Applicazione deve utilizzare il sistema ARPA per l'autenticazione e il recupero del token JWT
- I microservizi e le risorse REST dell'Applicazione devono essere mediati dall'API Gateway RT
- I singoli microservizi sviluppati devono necessariamente rispettare le regole di utilizzo della memoria descritte nel paragrafo precedente. Casi particolari di microservizi che necessitano di più RAM saranno analizzati e valutati insieme ai referenti regionali separatamente.



- Ogni singolo microservizio deve essere esposto su un contesto applicativo configurabile tramite configurazione esterna in modo da poterlo eventualmente modificare se necessario.
- Il container creato non deve esporre porte verso l'esterno. Infatti, i servizi REST implementati all'interno del microservizio sono esposti verso l'esterno direttamente dalla piattaforma Cochise2 utilizzando il contesto applicativo configurato.
- I servizi REST devono essere esposti sulla porta interna 9091; mediante la configurazione Kubernetes saranno poi esposti sulla porta 443 (protocollo https) direttamente dall'infrastruttura Cochise2.
- Ogni microservizio deve esporre obbligatoriamente un endpoint sul contesto <contesto-applicativo>/health che indichi lo stato di salute del microservizio stesso. Con <contesto-applicativo> si intende la variabile definita in configurazione come indicato sopra. Questo endpoint è utilizzato dal gestore per verificare se il deploy del microservizio è stato effettuato correttamente (Ad esempio: /cochise-api/app/health)
- Ogni microservizio può esporre metriche di monitoraggio applicativo. E' necessario, a tal proposito, esporre un endpoint sul contesto <contesto-applicativo>/metrics con le metriche di monitoraggio applicativo del singolo microservizio. Il formato di esposizione deve essere compatibile prometheus. Con <contesto-applicativo> si intende la variabile definita in configurazione come indicato sopra. Tali informazioni saranno recuperate mediante strumenti appositi installati sul cluster della piattaforma Cochise2 e visualizzabili in dashboards apposite rese disponibili ai fornitori.
- Non si può utilizzare servlet container come tomcat nei microservizi sviluppati in Java. In questo caso i microservizi devono utilizzare un servlet container "leggero" tipo Undertow. Inoltre, il jar relativo al microservizio deve essere autoconsistente, quindi comprensivo di tutte le librerie necessarie al funzionamento. Rimane valido il requisito iniziale che indica, laddove non ci siano controindicazioni giustificabili, di implementare i microservizi utilizzando linguaggi a basso consumo di risorse (Golang).
- Il container con il software deve poter essere dispiegabile in multi-ambiente, ossia in ambiente di certificazione che di produzione mediante la stessa immagine; il container deve accedere alla configurazione esterna attraverso variabili di ambiente configurabili in fase di deployment del servizio. Tale specifica è descritta nell'apposito documento ("Linee guida gestione configurazioni su K8s")
- Il microservizio nel container non deve creare file di log. Deve, invece, fare in modo che ogni processo di cui è composto si occupi di scrivere il proprio log di



eventi su “stdout” (ad esempio utilizzando appender di tipo “console”). Lo stdout viene poi catturato dall’ambiente di esecuzione kubernetes e inviato verso uno o più endpoint per l’archiviazione a lungo termine e la visualizzazione tramite gli strumenti messi a disposizione dalla piattaforma Cochise2. È fortemente consigliato, comunque, loggare le informazioni più rilevanti relative al microservizio in fase di startup per capire eventuali problematiche di deploy. Ulteriori informazioni relative ai pattern e alle specifiche di dettaglio dei log saranno fornite nelle linee guida di sviluppo.

- Per la visualizzazione dei log applicativi sarà predisposta una utenza applicativa dedicata al fornitore dell’applicazione. Mediante tale utenza sarà possibile visualizzare i log dell’applicativo deployato sulla piattaforma Cochise 2.

Per semplicità, nella tabella seguente riepiloghiamo le regole principali sopra elencate:

Regola	Impostazione
Front End	nativo su servizi/lavoro o formazione.toscana.it o altro front end
API Gateway	I microservizi dell’Applicazione devono essere mediati dall’API Gateway RT
Autenticazione	L’Applicazione deve utilizzare il sistema ARPA per l’autenticazione e il recupero del token JWT
ContextPath Backend	variabile appContextPath configurabile da file di configurazione
Linguaggio consigliato per lo sviluppo Backend	Golang per il basso utilizzo di risorse
Max RAM per singolo microservizio	Vedere tabella apposita
Porta interna di ascolto microservizio	9091
Servlet engine Java	Undertow (vietato Tomcat)
Informazioni di monitoraggio	esporre endpoint di health e un endpoint metrics per le metriche di monitoraggio applicativo
Configurazioni	su file esterno centralizzato
Log applicativi	su appender “console” senza scrittura di



	file
--	------

3. Monitoraggio e Logging

La Piattaforma Cochise mette a disposizione un logging centralizzato che consente di raccogliere i log da tutti i microservizi di un'Applicazione e ne facilita l'analisi e il debugging. Il logging centralizzato viene realizzato sulla piattaforma Cochise2 in modo trasparente al fornitore che può, quindi, concentrarsi solo sullo sviluppo dei singoli microservizi. In generale, come già evidenziato nella tabella del capitolo precedente, è sufficiente che il singolo microservizio effettui il log applicativo utilizzando lo standard output senza scrivere su file di log. A livello infrastrutturale, poi, tali log sono raccolti e inviati automaticamente alla componente centrale per l'analisi. Di seguito sono indicati alcune informazioni obbligatorie da inserire nel logging dei microservizi:

- Timestamp, livello di severità (info, warn, error), contesto dell'applicazione (ad esempio, ID della richiesta o del servizio), e dettagli dell'eventuale errore;
- Livelli di log: È consigliabile configurare i livelli di log per adattarsi agli ambienti (ad esempio, livelli di log più verbosi in ambienti di sviluppo e livelli ridotti in produzione).

3.1. OpenTelemetry

OpenTelemetry (<https://opentelemetry.io/>) è un progetto della Cloud Native Computing Foundation (CNCF), ed è utilizzato per la raccolta delle metriche di monitoraggio e tracciamento. Fornisce librerie utilizzabili per i principali linguaggi di sviluppo, tra cui Golang e Java. È fortemente consigliato integrare OpenTelemetry nello sviluppo delle Applicazioni per rendere disponibili, per ogni singolo microservizio, le metriche di monitoraggio applicative. Tali metriche, tramite gli strumenti messi a disposizione dalla piattaforma Cochise2, potranno alimentare opportune dashboard applicative, utili per il monitoraggio in real-time delle applicazioni e dei relativi microservizi sviluppati.

Il formato di esposizione dei dati di monitoraggio applicativo deve essere compatibile con Prometheus. A tal proposito, le librerie di OpenTelemetry da utilizzare per i linguaggi applicativi forniscono già un exporter compatibile con prometheus.

Di seguito alcune librerie per i principali linguaggi di programmazione:

Golang (Linguaggio da utilizzare per lo sviluppo delle nuove applicazioni):



Esistono librerie in grado di generare automaticamente metriche di monitoraggio quali ad esempio:

- numero di chiamate http agli endpoint esposti dal singolo microservizio
- utilizzo della ram allocata;
- latenza sulle richieste http.

4. Gestione aperture firewall di servizi esterni e Database

Al momento del dispiegamento e nei successivi deploy, eventuali connessioni esterne necessarie al servizio devono essere comunicate al supporto Cochise per permettere di effettuare verifiche preliminari.

Il Supporto Cochise invierà a SCT le eventuali richieste di aperture firewall tra la piattaforma evoluta Cochise 2 ed i servizi-esterni/database di riferimento dell'applicazione mettendo in cc il referente RT dell'applicazione e i referenti RT della piattaforma Cochise 2.

La comunicazione tra microservizi diventa fondamentale per garantire l'integrazione, l'interoperabilità e l'efficienza operativa all'interno di infrastrutture tecnologiche complesse. Di seguito sono descritti i modelli di comunicazione sincrono e asincrono utilizzabili nelle comunicazioni tra applicazioni e/o microservizi che espongono API REST.

5. Gestione comunicazioni asincrone

La comunicazione tra microservizi diventa fondamentale per garantire l'integrazione, l'interoperabilità e l'efficienza operativa all'interno di infrastrutture tecnologiche complesse. Di seguito sono descritti i modelli di comunicazione sincrono e asincrono utilizzabili nelle comunicazioni tra applicazioni e/o microservizi che espongono API REST.

5.1. Modelli di comunicazione: sincrono e/o asincrono

Un aspetto cruciale nella comunicazione tra sistemi è la distinzione tra modelli sincroni e asincroni. Questi modelli influenzano direttamente l'esperienza utente, le performance e la complessità delle applicazioni.



Comunicazione sincrona: In un modello sincrono, il client invia una richiesta al server e attende una risposta prima di proseguire con ulteriori elaborazioni. Questo tipo di comunicazione è comune nelle architetture client-server tradizionali e nelle API basate su http(s). Il vantaggio principale è la semplicità di implementazione e la linearità nella gestione delle risposte, ma può risultare inefficiente in termini di latenza, soprattutto in ambienti con alta richiesta.

Esempi: La maggior parte delle chiamate API REST sono sincrone. Un client invia una richiesta e rimane in attesa fino a quando il server risponde.

Svantaggi: Se il server impiega troppo tempo a rispondere, il client può restare bloccato, causando un deterioramento dell'esperienza utente o dei tempi di esecuzione.

Comunicazione asincrona: In un modello asincrono, il client invia una richiesta e prosegue le sue operazioni senza attendere immediatamente una risposta. Il server risponde in un secondo momento, notificando il client tramite un callback o un messaggio. Questo approccio è particolarmente utile in applicazioni che richiedono alte performance e in scenari in cui le operazioni possono richiedere molto tempo.

Sulla piattaforma Cochise2 sono presenti i seguenti broker per la gestione delle comunicazioni asincrone:

Nats Server

Apache Kafka

Se un'applicazione deve utilizzare un modello di comunicazione asincrono, sulla piattaforma Cochise è obbligatorio utilizzare necessariamente uno dei due broker presenti.

Entrambi i broker sono utilizzabili sotto autenticazione la cui configurazione deve essere richiesta al Supporto Cochise. I broker sono installati internamente alla piattaforma e non esposti per opportune ragioni di sicurezza.

6. Primo deploy su Cochise2

Il primo deploy di un nuovo microservizio su Cochise2 deve essere fatto mediante richiesta al Supporto Cochise. Tutti i deploy successivi, se viene abilitato l'autodeploy, potranno essere fatti in autonomia dal Fornitore.

È importante sottolineare alcuni aspetti fondamentali del primo deploy che devono essere forniti al Supporto Cochise:



- Il microservizio che viene deployato su Cochise2 deve essere “contestualizzato”, ossia deve essere classificato in base ad un ambito di appartenenza (lavoro, formazione, istruzione, etc)
- Il nome del microservizio deve essere “legato” all’applicazione mediante un prefix in modo tale da avere quanto più possibile nomi univoci all’interno della piattaforma.
- NON si accettano nomi generici dal momento che sulla piattaforma sono ospitate diverse applicazioni ed è quindi fondamentale classificare e contestualizzare i servizi da deployare (si veda i due punti precedenti)
- Il nome del microservizio, se si tratta di un back end, deve avere il suffisso finale “-ms”
- Il primo deploy deve essere autorizzato dal referente RT della piattaforma e deve essere richiesto dal Fornitore dell’Applicazione mediante ticket al Supporto Cochise

Esempio:

Per l’applicazione APP composta dai microservizi m1, m2, m3 i deploy da richiedere sono:

- app-m1-ms
- app-m2-ms
- app-m3-ms

7. Autodeploy su piattaforma Cochise2

La nuova piattaforma Cochise2, basata su Kubernetes, mette a disposizione un’opportuna API Rest utilizzabile per l’aggiornamento autonomo dei microservizi di un’applicazione.

Tale API Rest sarà integrata nella piattaforma OSCAT e resa disponibile al Fornitore.

Di seguito viene descritto il flusso per l’attivazione del servizio di autodeploy su Cochise2:

- Il primo deploy deve essere autorizzato dal referente RT della piattaforma e deve essere richiesto dal Fornitore dell’Applicazione mediante ticket al Supporto Cochise
- I deploy successivi (in ambiente di stage), invece, potranno essere effettuati in totale autonomia dal Fornitore mediante il servizio di autodeploy integrato con la piattaforma Oscat.
- Il Fornitore richiede l’attivazione del servizio di autodeploy al supporto Oscat



- Il Supporto OSCAT contatta il Supporto Cochise per ottenere le informazioni per attivare l'autodeploy
- Il Supporto Cochise restituisce le informazioni necessarie al Supporto Oscat per l'attivazione dell'autodeploy
- Il Supporto Oscat attiva l'autodeploy
- Il Fornitore, da questo momento, a seguito di una build su Oscat può eseguire il deploy in totale autonomia

7.1. Implementazione e Pubblicazione immagini sul registry evoluto

Nella soluzione PaaS viene offerto un Registry delle immagini arricchito ed evoluto, in gestione IaaS. L'implementazione di tale registry viene fatta utilizzando un prodotto open source presente sul mercato denominato Harbor.

Il Registry è fortemente integrato da una parte con la piattaforma OSCAT, che produce le immagini progettuali e le pubblica sul Registry, e dall'altra è integrato con il cluster Cochise che recupera le immagini da utilizzare per l'attivazione dei servizi sulla piattaforma.

Il Registry gestisce e mantiene le seguenti tipologie di immagini:

- progettuali: sono le immagini prodotte dai fornitori che sviluppano le Applicazioni;
- libreria RT: immagini di base personalizzate, gestite dal RTI e certificate che devono essere utilizzate per la costruzione delle immagini progettuali. All'interno della libreria sono presenti almeno le seguenti sottocategorie di immagini:
 - Infrastrutturali: sono le immagini relative ai servizi infrastrutturali (nats server, apache-kafka, etc) utilizzate per l'installazione di servizi infrastrutturali quali Apache Kafka e Nats Server per lo sviluppo di Applicazioni;
 - Sistemi operativi: immagini per il runtime del singolo microservizio relativo all'applicazione (distroless, ubuntu, nginx, etc);
 - Linguaggi di sviluppo: immagini di base specializzate per il linguaggio di programmazione da utilizzare per lo sviluppo del microservizio.

La struttura interna del registry che ospita le immagini è definita in modo tale che l'eventuale accesso sia limitato alle sole immagini progettuali delle proprie applicazioni e alle immagini della libreria RT.



Il set di immagini comprende tutte le immagini prodotte da utilizzare per la costruzione delle immagini progettuali delle varie Applicazioni. La realizzazione di tali immagini sarà basata su repository ufficiali e basandosi su versioni slim (come, ad esempio, le immagini alpine e/o distroless).

La pipeline e/o il processo di build di Oscan, quindi, integra il Registry evoluto della piattaforma in modo tale da poter pubblicare l'immagine che sarà utilizzata all'interno del cluster Cochise.

7.2. Naming convention per la produzione delle immagini di progetto

La Piattaforma Cochise è dotata di un registry personalizzato che contiene le immagini delle Applicazioni dispiegate sulla piattaforma.

Per poter gestire al meglio le immagini prodotte, di seguito viene definita la naming-convention da utilizzare per la produzione delle immagini a seguito della build della pipeline Oscan.

<https://<registry-cochise>/<ente>/<progetto>/<img-microservizio>:<versione>-<build>>

dove:

- registry-cochise è il nome del registry della Piattaforma Cochise 2 (sarà fornito dal Supporto Cochise)
- ente: ente di appartenenza dell'Applicazione
- progetto: nome dell'Applicazione
- img-microservizio: nome dell'immagine prodotta
- versione: versione dell'immagine relativa al microservizio
- build: numero progressivo della build prodotta dalla pipeline Oscan

Ad esempio:

<https://<registry-cochise>/giunta/nome-applicazione/app-ms:1.0.0-10>